

Agentic RFP Evaluation and Supplier Ranking

Build an AI-assisted application that reads supplier proposals, scores them against configurable criteria, benchmarks suppliers against their peers, and produces an explainable final leaderboard.

Application	Database	AI layer
Streamlit	SQLite	Any JSON-capable LLM

Learning goal

Understand how an agentic workflow can combine document reading, tool use, LLM reasoning, validation, deterministic business rules, database persistence, and an interactive user interface.

Project format: Individual | Submission timeline: 1 day

1. Business Problem

A procurement team receives several supplier RFP responses as PDF documents. Reading every proposal manually and comparing them consistently is slow. Your system will use AI to examine each proposal independently, while Python applies transparent scoring, peer comparison, tie-break rules, and ranking.

2. Minimum Functional Requirements

- Load active evaluation criteria and weights from SQLite and display them in Streamlit.
- Allow the user to upload multiple supplier RFP PDFs and enter supplier name, submission date, and historical experience rating.
- Create a batch and evaluate every supplier document against the latest active criteria.
- Extract PDF text and ask an LLM to return a criterion-wise score, justification, and supporting evidence in valid JSON.
- Validate and normalize missing, malformed, or out-of-range LLM results before scoring.
- Calculate absolute weighted scores, peer benchmarks, criterion gaps, relative percentages, and a weighted Peer Performance Index (PPI).
- Rank suppliers using the required deterministic tie-break order and store the complete result.
- Display a leaderboard and detailed scorecards; allow the complete result to be downloaded as JSON.

3. Agentic Design

Component	Responsibility	Suggested tools
Orchestrator Agent	Controls the workflow and calls the required tools in order.	Python functions or an agent framework
Document Tool	Extracts clean text from each uploaded PDF.	PyMuPDF / pypdf
Evaluation Agent	Uses active criteria to score one supplier and cites evidence.	LLM with structured JSON output
Validation Tool	Checks schema, fills missing criteria, clips invalid scores, and records warnings.	Pydantic / custom Python
Ranking Tool	Performs formulas, peer comparison, tie-breaks, and final ranking.	Deterministic Python only

Important: the LLM may judge proposal content, but it must not decide the final arithmetic, benchmark, tie-breaks, or rank.

4. Required Architecture and Data Flow

1	Setup Start Streamlit; load active criteria from SQLite.
2	Input Upload synthetic supplier PDFs and enter supplier metadata.
3	Batch Create supplier entries; generate a batch/run identifier.
4	Evaluate Reload criteria; extract text; build prompt; call the LLM.
5	Validate Parse JSON; normalize missing or invalid criterion results.
6	Score Calculate the absolute weighted score for every supplier.
7	Benchmark Find the best score per criterion and calculate peer metrics.
8	Rank Calculate PPI; apply tie-break rules; assign sequential ranks.
9	Persist Write complete results to SQLite using one RFP_RUN_ID.
10	Present Show leaderboard, evidence, comparisons, and JSON download.

5. Evaluation Criteria and Scoring

Store criteria in the database so the instructor or user can activate, deactivate, or change weights without changing the prompt code. The weights of active criteria should total 100%.

Example criterion	Weight	What the LLM should inspect
Technical Capability	30%	Architecture, integrations, scalability, technical fit
Implementation Plan	20%	Timeline, milestones, staffing, risk plan
Commercial Value	20%	Pricing clarity, total cost, assumptions
Security & Compliance	20%	Controls, certifications, privacy, auditability
Support & Experience	10%	Support model, similar projects, references

Suggested formulas

Absolute weighted score	Sum of (criterion score / maximum score) x criterion weight
Criterion benchmark	Highest valid score observed for that criterion across all suppliers
Criterion gap	Supplier score - benchmark score (zero for the benchmark leader; otherwise negative)
Relative performance %	(supplier score / benchmark score) x 100; define safe handling when benchmark is zero
Peer Performance Index	Weighted average of criterion relative-performance percentages

Mandatory tie-break order

1) Higher PPI first -> 2) Earlier submission date -> 3) Higher historical experience rating -> 4) Supplier name in ascending order. Assign rank 1, 2, 3... only after this stable sort.

6. Minimum SQLite Design

Table	Minimum fields
evaluation_criteria	criterion_id, name, description, weight, max_score, is_active
rfp_runs	rfp_run_id, created_at, status
supplier_results	rfp_run_id, supplier_name, submission_date, experience_rating, absolute_score, ppi, final_rank, result_json

7. Create Artificial RFP Documents

Create at least four fictional supplier PDFs for the same procurement request. Each proposal should be 2-4 pages and intentionally have different strengths, weaknesses, prices, schedules, evidence quality, and missing information. Do not use real confidential supplier data.

Supplier	Synthetic profile to include
Apex Systems	Strong technical design and security; higher price; moderate delivery schedule.
BrightPath Tech	Lowest price and fast timeline; weak compliance detail and limited experience.
NexaWorks	Balanced proposal; strongest implementation plan and support model.
Orbit Digital	Strong experience and references; vague integration plan; medium pricing.

Each artificial proposal should contain

- Executive summary and understanding of the requirement
- Proposed solution and implementation approach
- Timeline, team structure, and milestones
- Price table with assumptions
- Security, compliance, and risk controls
- Support model, relevant experience, and references

8. Expected LLM Output

```
{
  "supplier_name": "Apex Systems",
  "criteria": [{
    "criterion_id": 1, "score": 8, "max_score": 10,
    "justification": "...", "evidence": "..."
  }],
  "risks": ["..."], "overall_summary": "..."
}
```

Prompt requirement: tell the model to use only evidence present in the supplier document, return one result for every active criterion, stay within the score range, and output JSON only.

9. Streamlit Screens

Screen / section	What it must show
Criteria	Active criteria, weights, and maximum score
Supplier input	Multiple PDF upload, supplier metadata, validation messages, Evaluate button
Leaderboard	Rank, supplier, absolute score, PPI, submission date, experience rating
Detailed scorecard	Criterion score, benchmark, gap, relative %, weight, evidence, justification
Run details	RFP_RUN_ID, warnings, tie-break explanation, JSON download

10. Submission Requirements

- Complete source code with a clear folder structure and requirements.txt.
- SQLite database creation/seed script with sample evaluation criteria.
- At least four artificial supplier RFP PDFs.
- Deploy the complete working application on Streamlit Community Cloud and submit the public application URL.
- README with setup steps, architecture, formulas, assumptions, and screenshots.
- Sample exported JSON for one completed RFP run.
- A short demonstration showing one successful run and at least one validation/error case.

11. Evaluation Rubric (100 Marks)

Area	Marks	Assessment focus
Agentic workflow & tool use	20	Clear orchestration; appropriate separation of LLM and tools
PDF extraction & prompting	15	Reliable extraction, dynamic criteria, evidence-grounded JSON
Validation & scoring	20	Schema checks, normalization, correct weighted calculations
Peer ranking & tie-breaks	20	Benchmarks, PPI, deterministic order, explainability
SQLite & persistence	10	Criteria and complete run results stored correctly
Streamlit UI	10	Usability, leaderboard, drill-down, JSON download
Documentation & testing	5	Setup clarity, synthetic data, edge cases, reproducibility

Success condition

The same inputs must always produce the same formulas and ordering once the LLM scorecards have been validated. Every final score must be traceable to a criterion, weight, supplier evidence, and business rule.